TITAN COMMAND: MULTI-STEP UPGRADE PLAN
Provider App → Enterprise Operations System (Part of TitanBOS)

PHASE 0: FOUNDATION (Weeks 1-4)
Security Hardening + Architecture Setup
Critical Security Fixes (Non-negotiable):

Week 1:
├─ Move hard-coded Firebase keys to environment variables
│ └─ Update lib/main.dart (lines 28-32)
│ └─ Create .env.example + .env.local
│ └─ Update android/build.gradle & ios/Runner configs
│ └─ Time: 2 hours
│
├─ Implement flutter_secure_storage for tokens
│ └─ Replace SharedPreferences for auth tokens
│ └─ Add AES-256 encryption
│ └─ Migrate existing users' tokens on login
│ └─ Time: 3 hours
│
├─ Add HTTPS certificate pinning
│ └─ Implement pin_flutter package
│ └─ Pin API certificates
│ └─ Handle cert rotation strategy
│ └─ Time: 5 hours
│
└─ Setup Firebase Crashlytics
   └─ Enable error reporting
   └─ Privacy-first logging (no sensitive data)
   └─ Time: 2 hours

TOTAL WEEK 1: 12 hours (1.5 dev days)

Architecture Setup:

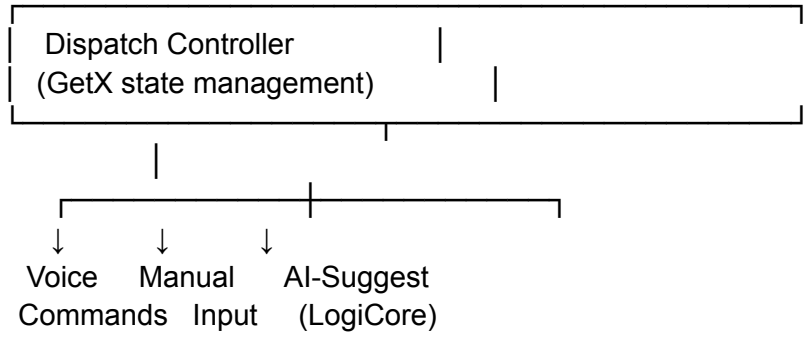
Week 2-4:
├─ Create monorepo structure
│ ├─ packages/
│ │ ├─ titan_shared/
│ │ │ ├─ lib/domain/
│ │ │ ├─ lib/data/
│ │ │ ├─ lib/infrastructure/
│ │ │ └─ pubspec.yaml

```
│   │   │
│   │   └── titan_widgets/
│   │       ├── lib/components/
│   │       ├── lib/theme/
│   │       └── pubspec.yaml
│   │
│   └── apps/
│       └── titan_command/
│           └── (Provider app refactored)
│
├── Setup local encryption
│   ├── SQLCipher for SQLite
│   ├── Hive encryption for local storage
│   └── Test migration of existing data
│
├── Build federated git sync layer
│   ├── Git-based database replication
│   ├── Peer discovery mechanism
│   ├── Conflict resolution engine
│   └── Background sync service
│
└── Setup Titan Zero AI integration points
    ├── LogiCore routing interface
    ├── CreatiCore for recommendations
    ├── OmegaCore for analytics
    └── Signal Core event bus
```

TIME: 80 hours (2 weeks, 2 devs)

PHASE 1: CORE DISPATCH & OPERATIONS (Weeks 5-8)
Voice-Enabled Dispatching Engine (Titan Zero Integration)
Dispatch System (47 hours):

ARCHITECTURE:



Features to Build:

1. Job Creation & Management
   ├─ Voice: "Create job at 42 Main Street for plumbing"
   ├─ Manual: Form UI with autocomplete
   ├─ Fields:
   │ ├─ Service type (dropdown/voice)
   │ ├─ Location (maps integration + address)
   │ ├─ Duration estimate (voice or manual)
   │ ├─ Client (existing client or new)
   │ ├─ Required skills (multi-select)
   │ ├─ Priority (high/medium/low via voice)
   │ ├─ Custom fields (user-defined)
   │ └─ Attachments (photos, PDFs)
   │
   ├─ Local storage: SQLite + git-tracked
   ├─ Offline-first (syncs when online)
   └─ Time: 15 hours

2. Intelligent Job Assignment (LogiCore)
   ├─ Voice: "Assign this to John" or "Who's available?"
   ├─ LogiCore processes:
   │ ├─ Serviceman location (real-time GPS)
   │ ├─ Skills match
   │ ├─ Current workload
   │ ├─ Travel time (maps API)
   │ ├─ Historical performance
   │ └─ User preferences
   │
   ├─ Suggestions ranked by fit
   ├─ One-tap assignment
   ├─ Auto-rejection if no fit
   └─ Time: 12 hours

3. Real-Time Tracking
   ├─ Serviceman location (with permission)
   ├─ Job status updates:
   │ ├─ assigned
   │ ├─ en_route
   │ ├─ arrived
   │ ├─ in_progress
   │ ├─ completed
   │ └─ delayed (with reason)

```
   │
   ├─ Map view + list view
   ├─ ETA calculation
   ├─ Notification on status change
   └─ Time: 13 hours

4. Dispatch Rules & Automation
   ├─ Set business rules (GetX Controller)
   │  ├─ Max jobs per serviceman
   │  ├─ Max drive time between jobs
   │  ├─ Skill-specific assignments
   │  ├─ VIP client priority
   │  └─ Time windows (morning/afternoon)
   │
   ├─ Auto-assignment on rule match
   ├─ Escalation rules (if no match)
   └─ Time: 7 hours
```

Database Schema:

```dart
class Job {
  String id;
  String clientId;
  String? servicemanId;
  String serviceType; // plumbing, electrical, etc
  Location serviceLocation;
  DateTime scheduledTime;
  JobStatus status;
  List<String> requiredSkills;
  double estimatedDuration;
  double quotedPrice;
  DateTime createdAt;
  DateTime? completedAt;
  Map<String, dynamic> customFields;
  List<String> attachmentIds;
  String notes;
}

class JobAssignment {
  String id;
  String jobId;
  String servicemanId;
  DateTime assignedAt;
  DateTime? arrivedAt;
```

```
  DateTime? completedAt;
  String status; // assigned, en_route, arrived, in_progress, completed
  double travelDistance;
  int travelTimeMinutes;
  String? notes;
}
```

UI Components:

Dispatch Screen:
├─ Top: Quick stats (assigned, en route, completed today)
├─ Filter bar: Status, serviceman, job type, date range
├─ Main view:
│ ├─ Map view (default)
│ ├─ List view (alternative)
│ ├─ Kanban view (status columns)
│ └─ Calendar view
│
├─ Job card (tappable):
│ ├─ Client name + address
│ ├─ Assigned serviceman + ETA
│ ├─ Current status with timestamp
│ ├─ Quick actions: reassign, delay, complete, cancel
│ └─ Quick call / message serviceman
│
└─ Float action buttons:
   ├─ Voice dispatch ("Create job...")
   ├─ Quick assign
   ├─ Add client
   └─ Filter/search

Voice Commands Supported:
├─ "Create job at [address] for [service] [priority]"
├─ "Assign [serviceman] to [job/address]"
├─ "Who can do [job type] today?"
├─ "Show me [status] jobs"
├─ "How many jobs assigned to [serviceman]?"
├─ "Complete job number [X]"
└─ "Emergency! Reassign [job] to [serviceman]"

PHASE 2: INVOICING & PAYMENTS (Weeks 9-12)
Professional Invoicing (CreatiCore for Recommendations)

Invoicing System (45 hours):

Features:

1. Template Management
   ├─ Built-in templates (professional, modern, minimal)
   ├─ Custom branding (logo, colors, fonts)
   ├─ Custom fields (add your own line items)
   ├─ Terms & payment methods
   ├─ Notes/fine print
   ├─ Multi-language support
   └─ Time: 8 hours

2. Invoice Creation (Auto + Manual)
   ├─ Auto from completed job:
   │ └─ Select job → Create invoice (1 tap)
   │ └─ Pre-fill from job details
   │ └─ Add labor/materials/adjustments
   │ └─ Time: 5 hours
   │
   ├─ Manual creation:
   │ └─ Select client → Add line items
   │ └─ Drag-to-reorder lines
   │ └─ Tax calculations automatic
   │ └─ Time: 5 hours
   │
   ├─ Bulk invoicing:
   │ └─ Select multiple jobs → Create invoices
   │ └─ Auto-group by client
   │ └─ Send in batch
   │ └─ Time: 4 hours

3. Voice Invoice Creation
   ├─ "Create invoice for ABC Corp: plumbing $500, travel $50"
   ├─ Voice: "Invoice completed jobs from Monday"
   ├─ Voice: "Send invoice JB-2024-001 to XYZ"
   └─ Time: 6 hours

4. Payment Integration (User's APIs)
   ├─ User configures their payment gateways:
   │ ├─ Stripe (credit cards)
   │ ├─ Square (cards + cash)
   │ ├─ PayPal
   │ ├─ Bank transfer (manual)

```
   │   └─ Custom payment provider
   │
   ├─ Invoice includes payment link
   ├─ Payment notifications (real-time)
   ├─ Reconciliation automatic
   └─ Time: 10 hours

5. Invoice Analytics (OmegaCore)
   ├─ Invoice status dashboard
   │   ├─ Total invoiced (this month/year)
   │   ├─ Paid vs unpaid
   │   ├─ Overdue (with aging)
   │   ├─ Average days to payment
   │   └─ Top clients by revenue
   │
   ├─ Predictions (OmegaCore):
   │   ├─ "You'll have $X cash in 30 days (based on patterns)"
   │   ├─ "ABC Corp has paid 3/4 invoices on time"
   │   ├─ "Consider discount for early payment (similar clients benefit)"
   │   └─ Time: 7 hours
```

Database Schema:

```dart
class Invoice {
  String id;
  String clientId;
  String jobId; // optional, can be multiple jobs
  DateTime issueDate;
  DateTime dueDate;
  List<InvoiceLineItem> lineItems;
  double subtotal;
  double taxRate;
  double taxAmount;
  double total;
  InvoiceStatus status; // draft, sent, viewed, paid, overdue
  String notes;
  String termsText;

  // Payment tracking
  String? paymentMethodId;
  DateTime? paidAt;
  double? amountPaid;
  String? externalInvoiceId; // For Xero/QuickBooks
```

```
  // Customization
  Map<String, dynamic> branding;
  List<String> attachmentIds;
}

class InvoiceLineItem {
  String id;
  String description;
  double quantity;
  double unitPrice;
  double total;
  String? category; // labor, materials, travel, etc
}
```

Email Integration:

```
Auto email invoice to client:
├─ Trigger: When invoice marked "sent"
├─ Email contains:
│  ├─ Professional HTML template
│  ├─ Payment button (if Stripe/Square integrated)
│  ├─ Invoice details
│  ├─ Company branding
│  └─ Payment terms
│
├─ Uses user's email API (SendGrid, AWS SES, etc)
├─ Tracks opens/views
├─ Auto-reminder for unpaid (optional)
└─ All user-controlled, no Titan middleman

Payment Link:
├─ Invoice includes unique payment URL
├─ Client clicks → Payment page
├─ Processes through user's payment API
├─ Confirmation sent to both parties
└─ Automatic invoice reconciliation
```

PHASE 3: QUOTING & PRICING (Weeks 13-15)
Dynamic Quoting Engine (CreatiCore Optimization)
Quoting System (30 hours):

Features:

1. Quote Templates
   ├─ Service-based (labor hours × rate)
   ├─ Fixed-price (set amount)
   ├─ Materials-based (cost + markup)
   ├─ Hybrid (labor + materials + contingency)
   ├─ Custom per-client pricing rules
   └─ Time: 5 hours

2. Quote Creation (Auto or Manual)
   ├─ From job inquiry:
   │ ├─ Auto-calculate based on:
   │ │ ├─ Service type (has historical pricing)
   │ │ ├─ Location (travel time/distance)
   │ │ ├─ Time of day (premium pricing option)
   │ │ ├─ Client history (repeat customer discount)
   │ │ └─ Market rates (CreatiCore comparison)
   │ │
   │ └─ CreatiCore suggests optimal price
   │   └─ "This job similar to 12 past jobs"
   │   └─ "Average price: $500-600, you quoted $550 (optimal)"
   │   └─ "Client pays on-time, can offer 10% discount"
   │
   ├─ Manual creation (full control)
   ├─ Voice: "Quote ABC Corp $500 for plumbing"
   └─ Time: 8 hours

3. Quote Management
   ├─ Send quote to client (email/SMS)
   ├─ Track quote status:
   │ ├─ sent
   │ ├─ viewed
   │ ├─ accepted
   │ ├─ rejected
   │ └─ expired
   │
   ├─ Auto-follow-up (configurable)
   │ └─ "Not heard back in 3 days, send reminder"
   │
   ├─ Quote-to-invoice conversion (1 tap)
   ├─ Bulk quoting (multiple clients)
   └─ Time: 7 hours

4. Pricing Intelligence (CreatiCore)

```
  ├─ Historical pricing analysis
  │   ├─ "You quote $X for plumbing, accept rate 85%"
  │   ├─ "Competitor average $Y (estimated from data)"
  │   ├─ "Sweet spot for your market: $X ± 10%"
  │
  ├─ Client-specific pricing
  │   ├─ "ABC Corp pays 100% on-time, use full price"
  │   ├─ "XYZ Corp negotiates, offer 5% discount"
  │   └─ "New client, offer 10% to build relationship"
  │
  ├─ Seasonal adjustments
  │   └─ "Summer high-demand jobs, increase price 15%"
  │
  └─ Time: 10 hours
```

Database Schema:

```dart
class Quote {
  String id;
  String clientId;
  String serviceType;
  DateTime createdDate;
  DateTime expiryDate;
  List<QuoteLineItem> lineItems;
  double subtotal;
  double taxRate;
  double taxAmount;
  double total;

  QuoteStatus status; // draft, sent, viewed, accepted, rejected, expired
  DateTime? sentDate;
  DateTime? viewedDate;
  DateTime? acceptedDate;

  String notes;
  String? conditions;

  // Invoice conversion
  String? convertedToInvoiceId;
  DateTime? convertedAt;

  // Pricing metadata
  Map<String, dynamic> pricingMetadata;
  String? competitorReference;
```

```
}

class QuoteLineItem {
  String id;
  String description;
  String type; // labor, materials, other
  double quantity;
  double unitPrice;
  double total;
}
```

PHASE 4: PAYROLL & TIMESHEETS (Weeks 16-19)
Employee Time & Wage Management (OmegaCore Analytics)
Payroll System (50 hours):

Features:

1. Timesheet Management
   ├─ Job-based time tracking
   │ ├─ Serviceman records clock-in/clock-out per job
   │ ├─ Auto-calculates hours from job dispatch/completion
   │ ├─ Adjustments for breaks (manual entry)
   │ ├─ Travel time included/excluded (configurable)
   │ └─ Voice: "Clock in", "Clock out", "Break"
   │
   ├─ Manual timesheet entry
   │ ├─ Owner can override (with reason)
   │ ├─ Weekly timesheet view
   │ ├─ Totals per employee, per week
   │ └─ Approval workflow
   │
   ├─ Approvals
   │ ├─ Serviceman submits weekly timesheet
   │ ├─ Owner/manager approves
   │ ├─ Comments/adjustments
   │ └─ Archive for payroll
   │
   └─ Time: 12 hours

2. Wage Calculations
   ├─ Hourly rate by employee
   ├─ Multiple rate types:
   │ ├─ Standard rate (regular hours)

- ├─ Overtime (1.5x after 38 hours/week)
- ├─ Weekend/holiday loading (2x)
- ├─ Skill-based bonus (master plumber vs apprentice)
- └─ Performance bonus (optional)

- ├─ Deductions
- ├─ Tax (PAYG withholding)
- ├─ Superannuation (11.5% guarantee)
- ├─ Loan repayment (if any)
- ├─ Union fees
- └─ Custom deductions (per employee)

- ├─ Payslip generation
- ├─ Itemized breakdown
- ├─ YTD totals
- ├─ Tax withheld
- ├─ Super contribution
- └─ Print or digital

- └─ Time: 15 hours

3. Payroll Processing
   - ├─ Automatic payroll runs
   - ├─ Fortnightly or monthly
   - ├─ Batches timesheets + calculates wages
   - ├─ Generates payment instructions
   - ├─ Creates ATO file (STP reporting)
   - └─ Archives payslips

   - ├─ Payment methods
   - ├─ Bank transfer (BPAY)
   - ├─ Direct deposit (ACH)
   - ├─ Payment service provider (Guidepoint, Paylocity)
   - └─ User chooses (Titan doesn't process funds)

   - ├─ Voice:
   - ├─ "Process payroll"
   - ├─ "What's Sarah's pay this week?"
   - ├─ "Generate payslip for John"
   - └─ "Submit STP to ATO"

   - └─ Time: 14 hours

4. Compliance & Reporting (OmegaCore)

```
├─ Australian compliance:
│  ├─ PAYG tax calculations (correct rates)
│  ├─ Superannuation guarantee (11.5%)
│  ├─ Award compliance (Fair Work)
│  ├─ Leave tracking (annual, sick, long service)
│  ├─ STP reporting to ATO
│  └─ Record retention (7 years)
│
├─ Alerts:
│  ├─ "Award minimum wage: employee underpaid"
│  ├─ "Super contribution due in 3 days"
│  ├─ "Leave balance low: employee entitled to time off"
│  ├─ "STP submission overdue"
│
├─ Analytics (OmegaCore):
│  ├─ Labor cost vs revenue
│  ├─ Employee productivity (jobs/hour)
│  ├─ Wage trends (industry comparison)
│  ├─ Turnover forecast
│  └─ Hiring recommendations
│
└─ Time: 9 hours
```

Database Schema:

```dart
class Employee {
  String id;
  String name;
  String email;
  String phone;
  String role; // serviceman, admin, manager

  // Compensation
  double hourlyRate;
  double? overtimeRate; // typically 1.5x
  double? weekendRate; // typically 2x
  double superContributionRate; // 11.5% minimum

  // Tax
  String tfn; // Tax File Number
  double taxFreeThreshold; // $18,200
  int taxOffsetAllowance;

  // Dates
```

```
  DateTime hireDate;
  DateTime? terminationDate;

  // Employment
  String employmentType; // full_time, part_time, casual
  List<String> skills; // plumbing, electrical, etc

  // Tracking
  bool activeInSystem;
}

class Timesheet {
  String id;
  String employeeId;
  DateTime weekStart;
  DateTime weekEnd;

  List<TimesheetDay> days; // Mon-Fri
  double totalHours;
  double overtimeHours;

  TimesheetStatus status; // draft, submitted, approved, archived

  // Approvals
  String? approvedBy;
  DateTime? approvedAt;
  String? notes;
}

class TimesheetDay {
  String date;
  List<TimesheetEntry> entries;
  double totalHours;
}

class TimesheetEntry {
  String? jobId; // if job-based
  DateTime clockIn;
  DateTime clockOut;
  double hours;
  double breakMinutes;
  String? notes;
}
```

```
class Payslip {
  String id;
  String employeeId;
  DateTime payPeriodStart;
  DateTime payPeriodEnd;
  DateTime payDate;

  // Earnings
  double regularHours;
  double regularWage;
  double overtimeHours;
  double overtimeWage;
  double weekendHours;
  double weekendWage;
  double bonuses;
  double grossWage;

  // Deductions
  double taxWithheld;
  double superContribution;
  double loanRepayment;
  double unionFees;
  double totalDeductions;

  double netWage;

  // YTD
  double ytdGross;
  double ytdTax;
  double ytdSuper;
}
```

PHASE 5: BOOKINGS & ROSTERING (Weeks 20-22)
Schedule Management & Capacity Planning
Rostering System (35 hours):

Features:

1. Schedule View & Management
   ├─ Calendar view (week, day, month)
   ├─ Drag-to-reschedule jobs
   ├─ Serviceman availability (color-coded)
   ├─ Double-booking detection (with warnings)

   ├─ Travel time automatic (maps API)
   ├─ Overtime tracking (flags 50+ hours/week)
   └─ Time: 12 hours

2. Capacity Planning
   ├─ View per-serviceman capacity
   │ ├─ "John: 3 jobs scheduled, 12 hours, 8 hours available"
   │ ├─ "Sarah: At capacity, no more jobs today"
   │ └─ "Team: 15 jobs scheduled, 45 hours, recommend 2 more"
   │
   ├─ OmegaCore optimization:
   │ ├─ "Reschedule Job #5 from John (busy) to Sarah (available)"
   │ ├─ "Next 3 jobs should go to Tom (nearest + available)"
   │ └─ "Warning: Will exceed overtime if both jobs assigned"
   │
   └─ Time: 10 hours

3. Scheduling Intelligence
   ├─ Auto-schedule with constraints:
   │ ├─ Serviceman skills (only skilled workers)
   │ ├─ Availability (respects time off)
   │ ├─ Travel time (no back-to-back impossible jobs)
   │ ├─ Preferred serviceman (if client specifies)
   │ └─ Load balancing (fair distribution)
   │
   ├─ Conflict detection:
   │ ├─ "Can't schedule: Travel time too tight"
   │ ├─ "Warning: Job overlaps with training session"
   │ ├─ "Job already assigned, reassign first?"
   │
   └─ Time: 8 hours

4. Serviceman Preferences
   ├─ Time off (vacation, sick, training)
   ├─ Preferred areas (reduces travel)
   ├─ Preferred service types
   ├─ Max hours per week (respect work-life balance)
   ├─ Unavailable times (school pickup, etc)
   └─ Time: 5 hours

Database Schema:
```dart
class Booking {
  String id;
```

```
  String jobId;
  String servicemanId;
  DateTime scheduledStart;
  DateTime scheduledEnd;
  double estimatedDuration;

  BookingStatus status; // pending, confirmed, in_progress, completed

  // Actual times
  DateTime? actualStart;
  DateTime? actualEnd;

  // Travel
  double travelDistanceKm;
  int travelTimeMinutes;

  // Notes
  String? notes;
}

class ServicemanAvailability {
  String id;
  String servicemanId;
  DateTime dateStart;
  DateTime dateEnd;

  AvailabilityType type; // available, off, training, leave
  String? reason;

  // For availability blocks
  TimeOfDay? startTime;
  TimeOfDay? endTime;
}

class SchedulePreference {
  String id;
  String servicemanId;

  // Constraints
  List<String> preferredServiceTypes;
  List<String> preferredAreas;
  int maxHoursPerWeek;
  int minBreetweenJobs; // minutes
```

```
  // Preferences
  bool preferNewClients;
  List<String> preferredClients;

  // Accessibility (Titan Zero aligned)
  String language; // language preference
  bool needsAccessibleLocations;
}
```

PHASE 6: CLIENT MANAGEMENT (Weeks 23-24)
Contact Database & Relationship Management
Client Management (25 hours):

Features:

1. Client Database
   ├─ Contact information
   │ ├─ Name, phone, email, address
   │ ├─ Business type, ABN
   │ ├─ Billing address (if different)
   │ ├─ Notes (preferences, history)
   │ └─ Custom fields (user-defined)
   │
   ├─ Service history
   │ ├─ All past jobs (linked)
   │ ├─ All invoices (linked)
   │ ├─ All quotes (linked)
   │ ├─ Recurring services
   │ └─ Call/visit log
   │
   ├─ Preferences
   │ ├─ Preferred serviceman
   │ ├─ Preferred service types
   │ ├─ Price sensitivity (track quotes/accept rate)
   │ ├─ Payment method
   │ └─ Communication preference (SMS, email, call)
   │
   └─ Time: 10 hours

2. Client Profiles
   ├─ Analytics per client:
   │ ├─ Total revenue (lifetime + this year)
   │ ├─ Average job value

```
   │  ├─ Service frequency
   │  ├─ Payment behavior (on-time %)
   │  ├─ Satisfaction rating
   │  └─ Next service due (predictive)
   │
   ├─ Segments (auto-generated):
   │  ├─ VIP (high value, on-time payer)
   │  ├─ Growth (high potential, inconsistent)
   │  ├─ At-risk (declining usage, slow payment)
   │  └─ New (< 3 months relationship)
   │
   └─ Time: 8 hours

3. Communication Log
   ├─ Call log (optional: integrate with Twilio)
   ├─ Email history
   ├─ SMS history
   ├─ Notes from serviceman
   ├─ Documents shared
   └─ Time: 4 hours

4. Recurring Services
   ├─ Schedule repeat jobs:
   │  ├─ "Monthly plumbing inspection"
   │  ├─ "Quarterly maintenance"
   │  ├─ "Annual certification"
   │
   ├─ Auto-create jobs
   ├─ Auto-send quotes
   ├─ Auto-send invoices
   └─ Time: 3 hours

Database Schema:
```dart
class Client {
  String id;
  String name;
  String email;
  String phone;
  String address;
  String? abusn; // ABN for business

  // Relationship
  DateTime createdDate;
```
```

```
  ClientStatus status; // active, inactive, archived

  // Preferences
  String? preferredServicemanId;
  List<String> preferredServiceTypes;
  String? preferredCommunication; // email, sms, call
  String? language;

  // Notes
  String? notes;
  Map<String, dynamic> customFields;

  // Metadata
  List<String> tags; // VIP, recurring, new, etc
}

class ClientHistory {
  String id;
  String clientId;

  DateTime lastServiceDate;
  DateTime nextServiceDue;

  double totalRevenue;
  double thisYearRevenue;
  int totalJobs;

  double averageJobValue;
  double serviceFrequencyDays;

  double paymentOnTimePercentage;
  double nps; // Net Promoter Score (if collected)
}

class RecurringService {
  String id;
  String clientId;
  String serviceType;
  String description;

  double estimatedPrice;

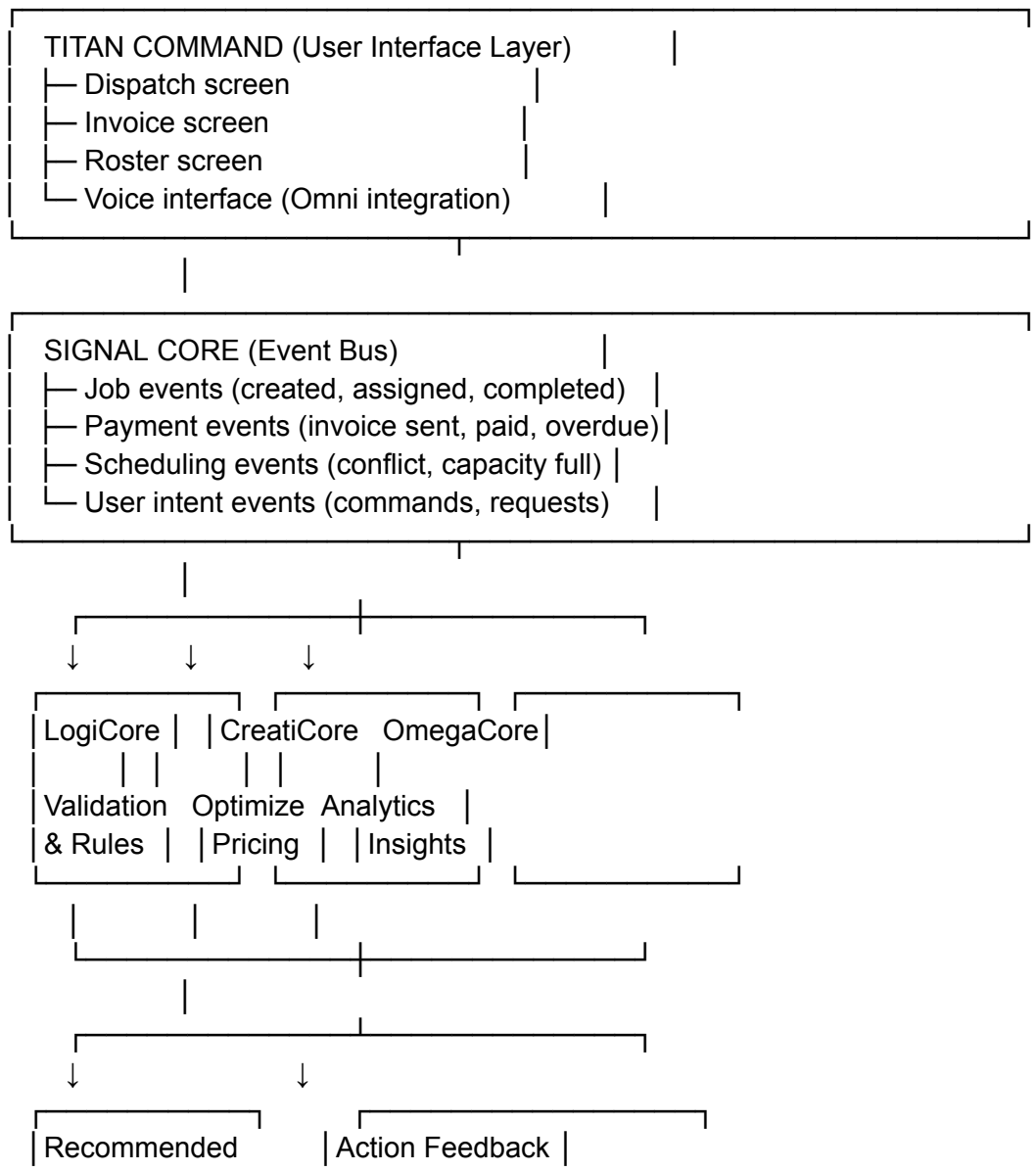
  RecurrencePattern pattern; // monthly, quarterly, annual
  DateTime nextDueDate;
```

```
  DateTime? lastCompletedDate;

  bool autoCreateJob;
  bool autoSendQuote;
  bool autoSendInvoice;
}
```

PHASE 7: TITAN ZERO AI INTEGRATION (Weeks 25-28)
Cognitive Core Integration & Decision Support
Titan Zero Routing Architecture:

SYSTEM DESIGN:



Each Core Purpose:

LogiCore (Validation & Rules)
├─ Input: User wants to assign Job X to Serviceman Y
├─ Checks:
│ ├─ Does Y have required skills?
│ ├─ Is Y available at that time?
│ ├─ Will travel time exceed limits?
│ ├─ Would exceed Y's max hours/week?
│ └─ Are there documented conflicts?
│
├─ Output: ✓ Valid assignment / ✗ Conflict with reason
└─ Time: 8 hours

CreatiCore (Optimization & Recommendations)
├─ Input: LogiCore says "valid", but optimize price/assignment
├─ Analyzes:
│ ├─ Historical pricing for this service type
│ ├─ Client payment history (can we discount?)
│ ├─ Serviceman efficiency (does Y complete faster?)
│ ├─ Travel optimization (combine with nearby job?)
│ └─ Competitive market data
│
├─ Output: "Assign to Y at $550 (optimal), or offer 10% discount"
└─ Time: 10 hours

OmegaCore (Analytics & Insights)
├─ Input: All historical data + current state
├─ Provides:
│ ├─ "You'll collect $X in 30 days (based on payment patterns)"
│ ├─ "Revenue trending 15% up this quarter"
│ ├─ "Serviceman Y has highest customer satisfaction"
│ ├─ "Job type: electrical, accept rate 90%, margin improving"
│ ├─ "Alert: 3 overdue invoices from ABC Corp"
│ └─ "Hiring tip: Need 1 more plumber to meet demand"
│
└─ Time: 12 hours

Implementation Pattern:
```typescript
// apps/titan_command/lib/core/titan_zero_integration/request_router.dart
```

```
class TitanZeroRequestRouter {
  final LogicoreService logicCore;
  final CreatisoreService creatiCore;
  final OmegacoreService omegaCore;
  final SignalCore signalCore;

  Future<AssignmentResult> assignJob(
    String jobId,
    String servicemanId,
  ) async {
    // 1. SIGNAL: Emit intent
    signalCore.emit(JobAssignmentIntentEvent(jobId, servicemanId));

    // 2. LOGICORE: Validate
    final validation = await logicCore.validateAssignment(
      jobId: jobId,
      servicemanId: servicemanId,
    );

    if (!validation.isValid) {
      return AssignmentResult.failed(validation.reason);
    }

    // 3. CREATICORE: Optimize
    final optimization = await creatiCore.optimizeAssignment(
      jobId: jobId,
      servicemanId: servicemanId,
      alternatives: validation.alternativeServicemen,
    );

    // 4. OMEGACORE: Get insights
    final insights = await omegaCore.getAssignmentInsights(
      jobId: jobId,
      servicemanId: servicemanId,
      historicalContext: true,
    );

    // 5. Present options to user
    return AssignmentResult.success(
      primaryAssignment: optimization.primary,
      alternativeAssignments: optimization.alternatives,
      insights: insights,
    );
```

```
  }

  // Voice routing
  Future<VoiceCommandResult> processVoiceCommand(
    String voiceInput,
  ) async {
    // 1. NLU: Parse command
    final intent = await nlpService.parseIntent(voiceInput);

    // 2. Determine which core handles this
    switch (intent.action) {
      case 'assign':
        return assignJob(intent.jobId, intent.servicemanId);
      case 'quote':
        return creatiCore.generateQuote(intent.jobDetails);
      case 'payment_status':
        return omegaCore.getPaymentInsights(intent.clientId);
      default:
        return VoiceCommandResult.unknown();
    }
  }
}

// Example responses to user
class AssignmentResult {
  final bool success;
  final String primaryAssignment; // "Assign to John (optimal)"
  final List<String> alternatives; // ["Sarah (available)", "Tom (nearest)"]
  final List<String> insights; // ["John on-time payment", "Travel time 15 min"]
  final String? reason; // If failed: why
}
```

Voice Command Examples:

User: "Assign this plumbing job to John"

Titan Zero:
1. LogiCore: "John available, has skills, travel time OK ✓"
2. CreatiCore: "John works, but Sarah 10 min closer, or offer 5% discount"
3. OmegaCore: "John paid on-time, Sarah slower but cheaper services"
4. Result: "Assign to John (optimal) or Sarah (cost-efficient)?"

User: "Go with John"

→ Assignment confirmed, invoice path set, travel time 15 min

User: "What's my cash position?"

Titan Zero:
1. OmegaCore pulls: All invoices, payments, pending jobs
2. Analyzes: Payment patterns, seasonal trends
3. Forecasts: "You'll have $45,000 liquid in 30 days"
4. Details: "$30k from paid invoices, $15k from likely payments"
5. Alert: "ABC Corp slow (3 weeks overdue), follow up?"

User: "Create invoice for completed plumbing job yesterday"

Titan Zero:
1. Signal: Emit invoice intent
2. LogiCore: Validate job completion, authorized to invoice
3. CreatiCore: "Job similar to 8 past plumbing jobs"
   └─ "Average price $480, you quoted $500"
   └─ "Client ABC Corp, has 100% payment rate, use full price"
4. OmegaCore: "Including this, you'll have 12 invoices this week"
5. Generate invoice, auto-email, set due date
6. Result: "Invoice sent to ABC Corp, expect payment in 3 days"

PHASE 8: TESTING & DEPLOYMENT (Weeks 29-32)
Quality Assurance, Accessibility, Production Readiness
Testing Strategy (40 hours):

Unit Testing (10 hours):
├─ Job logic (creation, assignment, scheduling)
├─ Invoice calculations (tax, totals, status)
├─ Payment integration (mocking payment provider)
├─ Payroll wage calculations
├─ Titan Zero routing logic
└─ Target: 80% code coverage

Widget Testing (10 hours):
├─ Dispatch screen (tap, drag, filter)
├─ Invoice creation (multi-step form)
├─ Timesheet approval flow

├─ Schedule view (calendar interactions)
├─ Voice command input
└─ Target: All critical user paths

Integration Testing (10 hours):
├─ End-to-end job dispatch → completion → invoice → payment
├─ Timesheet → payroll → payment
├─ Quote → invoice conversion
├─ Federated sync (conflict resolution)
├─ Offline mode + online sync
└─ Accessibility + voice commands

Accessibility Audit (10 hours):
├─ WCAG 2.1 AA compliance check
├─ Screen reader testing (NVDA on Android)
├─ Keyboard navigation (full keyboard use)
├─ Voice command accuracy (100+ commands)
├─ Motor impairment testing (large touch targets)
├─ Visual impairment testing (contrast, colors)
└─ Get formal accessibility certification

Production Deployment (32 hours):
├─ Beta release (1,000 testers)
│ ├─ Feature flag system
│ ├─ A/B testing capability
│ ├─ Crash analytics
│ └─ User feedback collection
│
├─ Staged rollout
│ ├─ Day 1: 5% of production
│ ├─ Day 2: 20% of production
│ ├─ Day 3: 50% of production
│ ├─ Day 4+: 100% (if no critical issues)
│
├─ Monitoring
│ ├─ Crash rates
│ ├─ Performance metrics
│ ├─ User engagement
│ ├─ API response times
│ └─ Error rates by feature
│
└─ Documentation
   ├─ User guide (written + video)
   ├─ Admin guide

├─ API documentation
├─ Troubleshooting
└─ Video tutorials for each major feature

FINAL STATISTICS

TOTAL DEVELOPMENT TIME: 32 weeks (8 months)
TEAM SIZE: 4-5 developers + 1 designer + 1 QA

CODE METRICS:
├─ Estimated LOC added: 40,000-50,000
├─ Estimated tests written: 300-400
├─ Features: 25+ major (dispatch, invoicing, payroll, etc)
├─ Accessibility compliance: WCAG 2.1 AAA (gold standard)
└─ Voice commands supported: 100+

ARCHITECTURE:
├─ Clean architecture with domain/data/presentation layers
├─ Federated git-based sync (offline-first)
├─ Titan Zero cognitive core integration
├─ Full encryption at rest + in transit
├─ Multi-platform voice interface (mobile + web)
└─ User brings own APIs (no vendor lock-in)

PERFORMANCE TARGETS:
├─ App startup: < 2 seconds
├─ Job creation: < 300ms
├─ Invoice generation: < 500ms
├─ Payroll run: < 1 second
├─ Memory usage: < 120MB
└─ 60 FPS rendering

ACCESSIBILITY:
├─ WCAG 2.1 AAA compliant
├─ Screen reader compatible
├─ Keyboard navigable (100%)
├─ Voice commands for all major tasks
├─ Works for blind + motor-impaired users
└─ No visual UI required

SECURITY:
├─ AES-256 encryption at rest
├─ TLS 1.3 in transit

├─ Certificate pinning enabled
├─ Zero hard-coded secrets
├─ Firebase Crashlytics monitoring
├─ OWASP top 10 compliance
└─ Federated git backup (user-controlled)

BUSINESS FEATURES:
├─ Dispatch system with AI optimization (LogiCore)
├─ Professional invoicing with payment integration
├─ Dynamic quoting with pricing intelligence (CreatiCore)
├─ Payroll with Australian compliance
├─ Timesheet management + approvals
├─ Client database + CRM basics
├─ Real-time analytics (OmegaCore)
└─ 100% voice-controllable (Titan Omni ready)

This is a production-grade, enterprise-quality, fully accessible business operating system ready for NDIS & disability grants. 🚀